1. Title

2. Project Goal

- Solve the Travelling Salesman Problem with a heuristic method.
- Parallelize the search with MPI on a multi-node cluster.
- Measure correctness, input-size behavior, granularity, load balance, and speedup.
- Keep the solver in C++; use Python only for visualization and plotting.

3. Travelling Salesman Problem

- Input: N cities with 2D coordinates.
- Output: one closed tour visiting every city exactly once.
- Objective: minimize total Euclidean route length.

L(T) = _i=0^N-1 d(t_i, t_(i+1) N)

TSP is NP-hard; exact brute force is only practical for very small N.

4. Repository Layout

Folder & Purpose \\
cpp/ & GA core, local search, MPI solver, tests \\
python/ & visualization, validation, experiments \\
data/ & city coordinate files and generator \\
cluster/ & OpenMPI, SSH, sync, launch scripts \\
results/ & CSV files, figures, generated outputs \\
report/ & report and presentation files \\

5. Main C++ Components

- ga_core.hpp: distance matrix, tour length, selection, OX crossover, mutation.
- local_search.hpp: 2-opt, Or-opt, optional memetic polish.
- tsp_island.cpp: MPI island-model solver.
- tsp_sequential.cpp: sequential baseline.
- test_*.cpp: unit tests for operators and local search.

6. Tour Representation

- One individual is a permutation of city indices.
- Example for 6 cities: [3, 0, 5, 1, 4, 2].
- The last city connects back to the first city.
- A valid tour must contain each index exactly once.

The fitness value is total tour length; smaller is better.

7. Sequential GA Baseline

- Generate an initial population of random tours.
- Evaluate all tour lengths.
- Keep the best individual by elitism.
- Select parents by tournament selection.
- Create children with order crossover.
- Apply swap and segment-reversal mutation.
- Repeat for a fixed number of generations.

8. Genetic Operators

- Tournament selection: choose the best among k=5 sampled individuals.
- Order crossover: preserve a segment from one parent and fill the rest using the order of another parent.
- Mutation:
- random city swap,
- random segment reversal.
- Elitism: copy the best local tour into the next generation.

9. Local Search

- Optional memetic step controlled by -twoopt.
- 2-opt removes edge crossings by reversing a segment.
- Or-opt moves a short segment to a better insertion position.
- Applied only periodically to the best local individual to limit cost.

10. Parallelism Level

Coarse-grained task-level parallelism.

- Each MPI rank runs one independent GA population.
- The city data and distance matrix are replicated on every rank.
- Different ranks use different random seeds and explore different regions.

Hybrid task and exploratory decomposition.

11. Why Exploratory Decomposition?

- A TSP tour is a global permutation, so splitting cities directly is not natural.
- GA search is stochastic; multiple populations can search different areas.
- Islands can run mostly independently and communicate only occasionally.
- This creates coarse granularity and reduces communication frequency.

12. Process Mapping

- One-dimensional mapping:
- Each rank owns one population and its fitness array.
- Cluster hostfile maps ranks round-robin across four machines.
- Up to 48 ranks were used, limited by the common physical-core count.

13. Communication Strategy

- Periodic blocking collective communication.
- Every -sync generations:
- each rank finds its local best,
- MPI_Allreduce(MINLOC) finds the global best and owner,
- owner broadcasts the best tour with MPI_Bcast,
- other ranks partially migrate this information into local populations.
- Logical topology: global collective, not ring and not master-slave.

14. Partial Global-Best Migration

- The owner broadcasts the best tour.
- Other islands do not clone the full tour.
- They use OX crossover between:
- the global best tour,
- a random local mate.
- The child replaces one of the worst local individuals.
- Goal: spread good sub-routes while preserving diversity.

15. Parallel Algorithm

Initialize MPI
Read cities and build distance matrix on every rank
Create one random population per rank
for each generation:
evolve local population
optionally polish local best
if generation is a sync point:
local_best = best local tour
global_best, owner = MPI_Allreduce(MINLOC)
owner broadcasts global_best tour
non-owner ranks inject partial migrants
update convergence-stop state
Gather final global best and write outputs
Finalize MPI

16. Load Balancing

- Default: same population size on every rank.
- This gives approximately equal compute work per rank.
- The cluster run limits ranks per machine by the weakest machine.
- Optional -auto-balance benchmarks local speed and gives faster ranks larger populations.

One GA population per process: coarse enough to avoid per-generation communication.

17. Instrumentation

- comm_s: measured time inside MPI calls.
- compute_s: estimated as total time minus communication time.
- makespan_s: maximum elapsed time over all ranks.
- -stats: writes one CSV row per rank.
- -out: writes final tour and convergence history.
- -live: streams JSONL for real-time visualization.

18. Correctness Validation

- Unit tests verify:
- distance matrix correctness,
- OX crossover produces valid permutations,
- mutation preserves permutations,
- local search does not worsen tours.
- validate_tour.py independently recomputes route length.
- For N 10, brute force gives the exact optimum for comparison.

19. Cluster Setup

- Four machines connected over a LAN.
- OpenMPI 5.0.9 pinned at /opt/openmpi-5.0.9.
- Password-less SSH for remote rank launch.
- Same repository path on every node.
- Launcher uses -map-by seq -bind-to none for heterogeneous nodes.

20. Experiment 1: Runtime vs Input Size

21. Final N* Selection

N & Total (s) & Comm (s) & Compute (s) \\

1200 & 54.54 & 25.90 & 28.65 \\

1800 & 86.99 & 34.33 & 52.66 \\

2400 & 154.27 & 64.05 & 90.22 \\

- Final choice: N^*=2400, gens*=10500, 48 ranks.
- Official repeated run: 145.97s, inside the 120-180s target.

22. Experiment 2: Granularity

23. Granularity Result

- Run configuration: N^*=2400, gens=10500, 48 ranks.
- Each rank owns one island population.
- The measured idle skew is about 1.8\
- This is below the 25\

The workload is balanced for this configuration.

24. Experiment 3: Speedup

[Figure: ../results/exp_speedup.png]

25. Speedup Result and Conclusion

- Speedup experiment uses 2N^*=4800 cities and fixed total population.
- Speedup improves up to 16 processes, then decreases.
- No-communication speedup is much higher than total speedup.
- The GA computation scales, but repeated collective communication dominates at high process counts.
- The island model is correct and balanced, but large process counts require larger problem sizes or less frequent synchronization.

Parallel level: task/exploratory. Mapping: 1D rank-to-island. Communication: blocking MPI collectives. Load balance: acceptable in the measured run.